

Leases & Escrow — Frontend Integration Guide

Companion to `LISTINGS_GUIDE.md` . Covers the full lease lifecycle: creation, proposal, tenant signing, on-chain escrow (fund → activate → settle), disputes, purchase transactions for sales, rental yield tracking, and the timeline API.

Table of Contents

- 1. [The Lease Object](#)
- 2. [Lease Lifecycle \(State Machine\)](#)
- 3. [Creating a Lease](#)
- 4. [Proposing & Signing](#)
- 5. [On-Chain Escrow — LeaseEscrow](#)
- 6. [Lease Settlement \(Complete / Terminate / Cancel\)](#)
- 7. [Disputes](#)
- 8. [Reading Leases](#)
- 9. [Escrow Info & Timeline](#)
- 10. [Tenant Roster](#)
- 11. [Purchase Transactions \(Sale Escrow\)](#)
- 12. [Rental Yield & Maintenance Records](#)
- 13. [Chain Transactions \(Audit Trail\)](#)
- 14. [Notifications](#)
- 15. [Error Reference](#)
- 16. [Recommended Frontend Patterns](#)

1. The Lease Object

Key fields returned by the API:

Field	Type	Notes
id	string	Unique lease identifier
listing	string	ObjectId of the linked listing (always <code>listingType=rent</code>)
landlord	string	ObjectId of the property owner
tenant	string	ObjectId of the tenant
monthlyRent	number	Monthly rent amount (≥ 0)
depositAmount	number	Security deposit (≥ 0)
escrowAmount	number	Auto-calculated: <code>monthlyRent + depositAmount</code>
currency	string	3-letter ISO code, uppercase (default <code>USD</code>)
startDate	ISO date	Lease start

Field	Type	Notes
endDate	ISO date	Lease end (must be after startDate)
terms	string?	Free-text lease terms (max 20,000 chars)
termsHash	string?	SHA-256 of canonical lease fields — set on propose
status	string	Current lifecycle state
escrow	object	On-chain escrow sub-document (see below)
signedByTenantAt	ISO date?	When the tenant signed
tenantSignature	string?	Optional signature string
dispute	object?	Dispute sub-document (see below)
createdBy	string	Who created the lease
createdAt / updatedAt	ISO date	Timestamps

Escrow sub-document

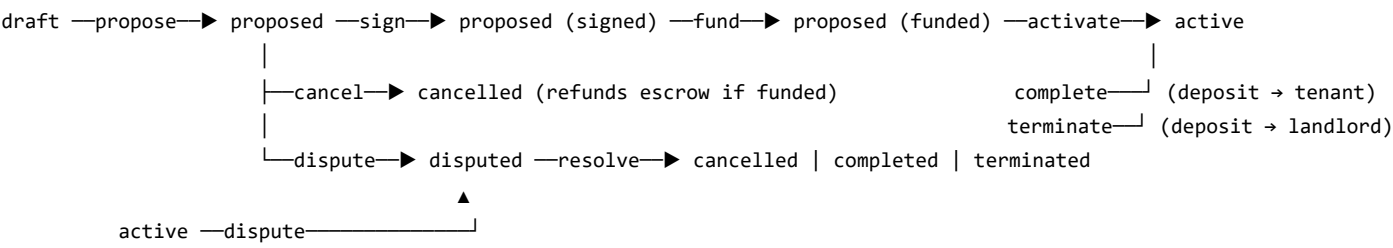
Field	Type	Notes
escrowId	string?	On-chain escrow ID (numeric string)
contractAddress	string?	LeaseEscrow contract address
token	string?	ERC-20 stablecoin address
state	string	"none" → "funded" → "active" → "closed"
fundTxHash	string?	Transaction hash of openAndFund
activateTxHash	string?	Transaction hash of activate
settleTxHash	string?	Transaction hash of final settlement
landlordWallet	string?	Landlord's Ethereum address
tenantWallet	string?	Tenant's Ethereum address

Dispute sub-document

Field	Type	Notes
openedBy	string?	Who opened the dispute
openedAt	ISO date?	When it was opened
reason	string?	Free-text reason (max 2,000 chars)
response	string?	Counterparty's response (max 2,000 chars)
respondedBy	string?	Who responded
respondedAt	ISO date?	When the response was recorded

2. Lease Lifecycle (State Machine)

Status flow



Lease statuses

Status	Description
draft	Initial state. Lease terms defined but not yet shared.
proposed	Landlord proposed the lease. Tenant can sign it. Escrow can be funded.
active	Escrow activated. First month's rent released to landlord. Deposit held.
completed	Lease ended normally. Deposit refunded to tenant .
terminated	Lease ended early. Deposit released to landlord .
cancelled	Lease cancelled before activation. If escrow was funded, full refund to tenant .
disputed	A dispute is open. Must be resolved by admin before settlement.

Transition rules

Action	Who	From	Pre-conditions
propose	landlord, admin	draft	Generates termsHash
sign	tenant, admin	proposed	Optional tenantSignature
fund	admin	proposed	Tenant signed + both parties KYC verified + both wallets linked
activate	admin	proposed	Escrow state = funded
cancel	landlord, tenant, admin	proposed	If escrow funded → on-chain refund
complete	admin	active	On-chain refundDeposit (deposit → tenant)
terminate	admin	active	On-chain releaseDeposit (deposit → landlord)
dispute	landlord, tenant, admin	proposed , active	Creates compliance case
dispute/respond	counterparty, admin	disputed	The opener cannot respond to their own dispute

Action	Who	From	Pre-conditions
dispute/resolve	admin	disputed	On-chain settlement based on decision

Key insight: The difference between complete and terminate is **who gets the deposit**. Complete = tenant gets deposit back. Terminate = landlord keeps deposit.

3. Creating a Lease

From a rental application (recommended flow)

The typical path is: Rental Application → Approved → Create Lease.

```
// Landlord creates lease from an approved rental application
await api(`/rental-applications/${applicationId}/lease`, {
  method: "POST",
  body: JSON.stringify({
    monthlyRent: 1500,
    depositAmount: 3000,
    currency: "USD",
    startDate: "2026-08-01",
    endDate: "2027-07-31",
    terms: "Standard 12-month lease agreement...",
  }),
});
```

- Application must be in approved status
- Creates the lease AND sets application status to lease_created

Direct creation (landlord or admin)

```
await api("/leases", {
  method: "POST",
  body: JSON.stringify({
    listingId: "64abc123def456789012abcd",
    tenantId: "64def456abc789012345efgh",
    monthlyRent: 1500,
    depositAmount: 3000,
    currency: "USD",
    startDate: "2026-08-01",
    endDate: "2027-07-31",
    terms: "Standard 12-month lease...",
  }),
});
```

Validation rules

- listingId must reference a **published, rent-type** listing
- tenantId must reference a valid user

- `endDate` must be **after** `startDate`
- `monthlyRent` and `depositAmount` must be ≥ 0
- `escrowAmount` is auto-calculated: `monthlyRent + depositAmount`
- **Overlap guard:** rejects if there's an existing non-terminal lease (`draft` , `proposed` , `active` , `disputed`) for the same listing whose dates overlap the requested period → returns `409`
- Only the listing owner (or admin) can create — others get `403`

4. Proposing & Signing

Step 1: Propose (landlord)

```
await api(`/leases/${id}/propose`, { method: "POST" });
```

- Transitions from `draft` → `proposed`
- Generates a `termsHash` (SHA-256 of canonical fields: listing, landlord, tenant, rent, deposit, dates, terms)
- Notifies both parties

Step 2: Sign (tenant)

```
await api(`/leases/${id}/sign`, {
  method: "POST",
  body: JSON.stringify({
    tenantSignature: "0x...", // optional, max 1000 chars
  }),
});
```

- Only the lease's tenant (or admin) can sign
- Sets `signedByTenantAt` timestamp
- Stores optional `tenantSignature` (e.g., a wallet signature for non-repudiation)
- The lease **stays in proposed status** — signing doesn't change the status, it records consent
- Notifies the landlord

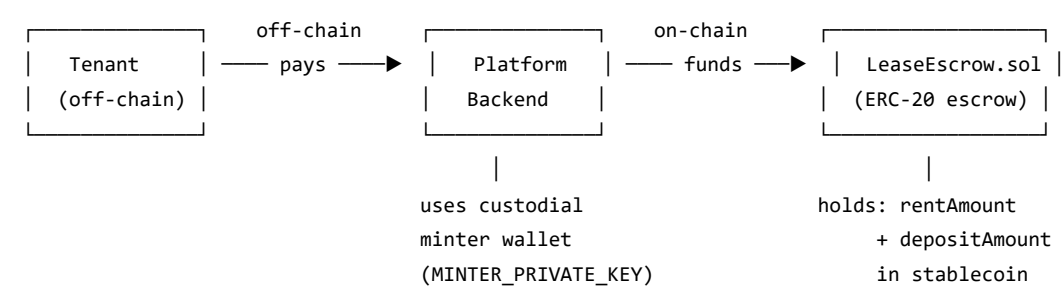
Frontend guidance

- After proposing, show the tenant a "Review & Sign" UI with lease terms
- Gray out the "Fund Escrow" admin button until `signedByTenantAt` is set
- Display the `termsHash` for transparency — both parties can verify the hash matches the terms they agreed to

5. On-Chain Escrow — LeaseEscrow

The platform uses a **custodial escrow model**: the platform's backend wallet holds and moves funds on behalf of landlord and tenant. No user directly interacts with the smart contract.

Architecture



What the escrow holds

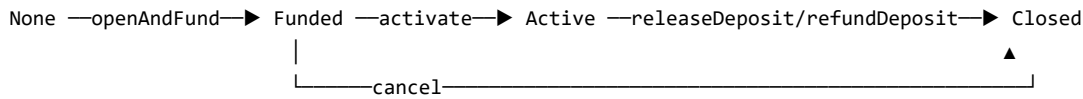
Component	Amount	Released when
First month's rent	monthlyRent	On activate → sent to landlord
Security deposit	depositAmount	On settlement → sent to landlord OR tenant

Total locked: `escrowAmount = monthlyRent + depositAmount`

Smart contract functions (for reference)

Contract method	Backend trigger	What happens
<code>openAndFund(leaseId, landlord, tenant, token, rent, deposit, termsHash)</code>	<code>POST /leases/:id/fund</code>	Pulls rent + deposit in stablecoin from platform wallet into escrow
<code>activate(escrowId)</code>	<code>POST /leases/:id/activate</code>	Releases first month's rent to landlord wallet
<code>refundDeposit(escrowId)</code>	<code>POST /leases/:id/complete</code>	Sends deposit back to tenant wallet
<code>releaseDeposit(escrowId)</code>	<code>POST /leases/:id/terminate</code>	Sends deposit to landlord wallet
<code>cancel(escrowId)</code>	<code>POST /leases/:id/cancel</code>	Refunds everything (rent + deposit) to tenant wallet
<code>getEscrow(escrowId)</code>	<code>GET /leases/:id/escrow</code>	Read-only: returns on-chain state

Escrow state machine (on-chain)



Fund escrow (admin only)

```
await api(`/leases/${id}/fund`, { method: "POST" });
```

Pre-conditions (all must be met, or returns error):

Condition	Error if unmet
Tenant has signed (signedByTenantAt set)	409
Escrow state is none	409 "Escrow already funded"
Both landlord and tenant have linked wallet addresses	400
Both landlord and tenant are KYC verified	403
Lease has a termsHash (was proposed)	409

What happens on success:

1. Backend calls `LeaseEscrow.openAndFund()` on-chain
2. Stablecoin tokens are pulled from the platform wallet into the escrow contract
3. Escrow sub-document is updated with `escrowId` , `contractAddress` , `token` , `state: "funded"` , `fundTxHash`
4. Both parties are notified
5. A `ChainTransaction` audit record is created

Activate (admin only)

```
await api(`/leases/${id}/activate`, { method: "POST" });
```

- Requires escrow state = funded
- Calls `LeaseEscrow.activate()` on-chain → first month's rent is released to landlord
- Lease status → active , escrow state → active
- Both parties notified

6. Lease Settlement (Complete / Terminate / Cancel)

Complete — deposit refunded to tenant

```
await api(`/leases/${id}/complete`, { method: "POST" });
```

- Admin only. Lease must be active .

- Calls `LeaseEscrow.refundDeposit()` → deposit goes to **tenant** wallet
- Status → `completed` , escrow state → `closed`
- Use when: lease ends normally, no damages

Terminate — deposit released to landlord

```
await api(`/leases/${id}/terminate`, { method: "POST" });
```

- Admin only. Lease must be `active` .
- Calls `LeaseEscrow.releaseDeposit()` → deposit goes to **landlord** wallet
- Status → `terminated` , escrow state → `closed`
- Use when: early termination, damages, or lease breach

Cancel — full refund to tenant

```
await api(`/leases/${id}/cancel`, { method: "POST" });
```

- Any party (landlord, tenant, admin). Lease must be `proposed` .
- If escrow was funded: calls `LeaseEscrow.cancel()` → **rent + deposit** refunded to tenant
- If escrow was not funded: simply cancels the lease
- Status → `cancelled` , escrow state → `closed`

Settlement summary

Endpoint	From status	Deposit goes to	Rent goes to
<code>complete</code>	<code>active</code>	Tenant	Already released to landlord on activate
<code>terminate</code>	<code>active</code>	Landlord	Already released to landlord on activate
<code>cancel</code>	<code>proposed</code>	Tenant (if funded)	Tenant (if funded)

7. Disputes

Open a dispute (any party)

```
await api(`/leases/${id}/dispute`, {
  method: "POST",
  body: JSON.stringify({
    reason: "Landlord failed to make agreed repairs", // optional, max 2000 chars
  }),
});
```

- Allowed by: landlord, tenant, or admin
- Lease must be in `proposed` or `active` status
- Status → `disputed`
- Creates a compliance case automatically (flagged for admin review)
- Both parties notified

Respond to a dispute (counterparty)

```
await api(`/leases/${id}/dispute/respond`, {
  method: "POST",
  body: JSON.stringify({
    response: "Repairs were completed on June 10th, photos attached via email",
    // required, max 2000 chars
  }),
});
```

- Only the **counterparty** (the party who did NOT open the dispute) or admin can respond
- The opener cannot respond to their own dispute → returns 403
- Notifies the dispute opener

Resolve a dispute (admin only)

```
await api(`/leases/${id}/dispute/resolve`, {
  method: "POST",
  body: JSON.stringify({
    decision: "refund_deposit", // "release_deposit" | "refund_deposit" | "cancel"
    note: "After review, tenant's complaint is valid", // optional
  }),
});
```

Decision outcomes:

Decision	Escrow required state	Result
cancel	funded (pre-activation)	Full refund to tenant. Status → cancelled
release_deposit	active	Deposit → landlord. Status → terminated
refund_deposit	active	Deposit → tenant. Status → completed

- Each decision triggers the corresponding on-chain escrow call
- If escrow state doesn't match the decision, returns 409

Frontend guidance for disputes

- Show a "Dispute" button on proposed and active leases
- After a dispute is opened, show the dispute reason and a response form for the counterparty
- Admins see a "Resolve Dispute" panel with three options and a note field
- Disable the response form for the party who opened the dispute

8. Reading Leases

List my leases

```
const leases = await api("/leases/mine");  
// Returns all leases where user is landlord OR tenant  
// Sorted by createdAt descending
```

- Authenticated. Any role with `property_owner`, `tenant`, `admin`, or `super_admin`.
- Returns leases from both sides (landlord and tenant perspectives)

Get single lease

```
const lease = await api(`/leases/${id}`);
```

- Only visible to lease parties (landlord or tenant) and admins
- Non-parties get 404 (not 403 — for privacy)

9. Escrow Info & Timeline

Get escrow info

```
const escrowInfo = await api(`/leases/${id}/escrow`);
```

Response shape:

```
{  
  "lease": { /* full lease object */ },  
  "onChain": {  
    "state": "active",  
    "landlord": "0xabc...",  
    "tenant": "0xdef...",  
    "rentAmount": "15000000000000000000",  
    "depositAmount": "30000000000000000000",  
    "termsHash": "sha256..."  
  }  
}
```

- `onChain` is `null` if no escrow has been funded yet
- `onChain` reads directly from the blockchain — it's the **source of truth**
- `rentAmount` and `depositAmount` are in token base units (e.g., 18 decimals for most ERC-20s)

Get timeline

```
const timeline = await api(`/leases/${id}/timeline`);
```

Response shape:

```
{
  "leaseId": "64abc...",
  "currentStatus": "active",
  "escrowState": "active",
  "events": [
    {
      "key": "created",
      "label": "Lease created",
      "at": "2026-06-01T10:00:00Z",
      "status": "completed"
    },
    {
      "key": "proposed",
      "label": "Lease proposed",
      "at": "2026-06-02T10:00:00Z",
      "status": "completed",
      "metadata": { "termsHash": "abc123..." }
    },
    {
      "key": "signed",
      "label": "Tenant signed",
      "at": "2026-06-03T10:00:00Z",
      "status": "completed"
    },
    {
      "key": "escrow_funded",
      "label": "Escrow funded",
      "at": "2026-06-04T10:00:00Z",
      "status": "completed",
      "metadata": { "txHash": "0x...", "escrowId": "1" }
    },
    {
      "key": "active",
      "label": "Lease activated",
      "at": "2026-06-05T10:00:00Z",
      "status": "completed",
      "metadata": { "txHash": "0x..." }
    },
    {
      "key": "settled",
      "label": "Lease settled",
      "at": null,
      "status": "pending",
      "metadata": { "txHash": null, "finalStatus": "active" }
    }
  ]
}
```

Event statuses: "completed" (done), "pending" (not yet), "active" (in progress — only for disputes).

If a dispute was opened, an extra event is appended:

```
{
  "key": "disputed",
  "label": "Dispute opened",
  "at": "2026-06-10T10:00:00Z",
  "status": "active",
  "metadata": { "reason": "..." }
}
```

Frontend guidance

- Render the timeline as a **vertical stepper** component
- Color each step: green (completed), blue (active/current), gray (pending)
- Show `metadata.txHash` as a clickable link to a block explorer (e.g., [https://sepolia.etherscan.io/tx/\\${txHash}](https://sepolia.etherscan.io/tx/${txHash}))
- Use `metadata.escrowId` to link to the on-chain escrow record

10. Tenant Roster

Landlords can view all their current and past tenants across all properties.

List tenants (landlord)

```
const roster = await api("/leases/tenants");
```

Response shape:

```
[
  {
    "leaseId": "64abc...",
    "tenant": {
      "id": "64def...",
      "name": "Abebe Kebede",
      "email": "abebe@example.com"
    },
    "listing": {
      "id": "64ghi...",
      "title": "Modern 3BR Apartment"
    },
    "startDate": "2026-08-01",
    "endDate": "2027-07-31",
    "status": "active",
    "monthlyRent": 1500
  }
]
```

Admin: view all tenants (with optional filter)

```
// All tenants across all landlords
const all = await api("/leases/tenants");

// Filter by specific landlord
const filtered = await api("/leases/tenants?ownerId=64abc...");
```

- `property_owner` role: sees only their own tenants
- `admin/super_admin` : sees all tenants, optionally filtered by `ownerId` query param

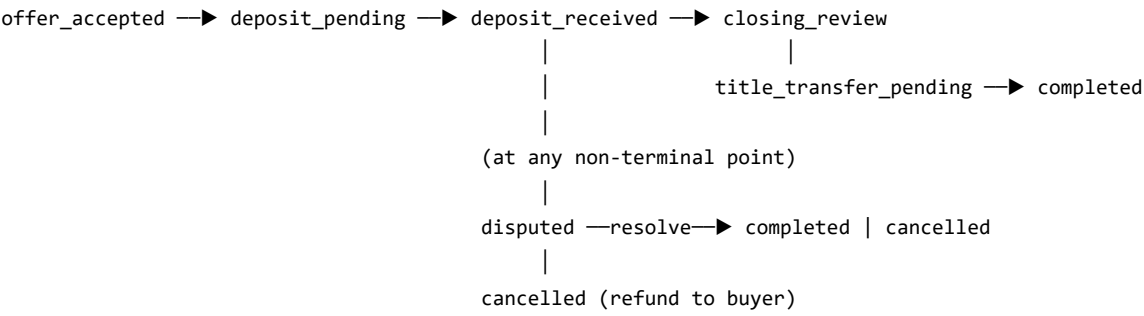
11. Purchase Transactions (Sale Escrow)

Purchase transactions track property sales from offer acceptance through escrow settlement and title transfer. They mirror the lease escrow pattern but use the `SaleEscrow.sol` contract with a simpler flow.

The Purchase Transaction Object

Field	Type	Notes
<code>id</code>	<code>string</code>	Unique ID
<code>listing</code>	<code>string</code>	<code>ObjectId</code> → Listing
<code>offer</code>	<code>string</code>	<code>ObjectId</code> → Offer (1:1 unique)
<code>seller</code>	<code>string</code>	<code>ObjectId</code> → User (listing owner)
<code>buyer</code>	<code>string</code>	<code>ObjectId</code> → User
<code>amount</code>	<code>number</code>	Sale amount (from accepted offer)
<code>currency</code>	<code>string</code>	ISO currency code
<code>status</code>	<code>string</code>	Current status (see below)
<code>depositAmount</code>	<code>number?</code>	Optional deposit amount
<code>escrow</code>	<code>object</code>	On-chain sale escrow sub-document
<code>termsHash</code>	<code>string?</code>	SHA-256 of purchase terms
<code>titleTransferTxHash</code>	<code>string?</code>	Title NFT transfer transaction hash
<code>closingChecklist</code>	<code>object</code>	5-item boolean checklist
<code>dispute</code>	<code>object?</code>	Dispute data
<code>timeline</code>	<code>array</code>	Timeline events

Purchase transaction statuses



Status	Meaning
offer_accepted	Transaction created from accepted offer
deposit_pending	Waiting for deposit from buyer
deposit_received	Escrow funded on-chain
closing_review	Admin reviewing closing checklist
title_transfer_pending	Title NFT being transferred to buyer
completed	Escrow released + title transferred
cancelled	Transaction cancelled, escrow refunded
disputed	Dispute open

How it starts

Purchase transactions are **auto-created** when an offer is accepted:

```
// Accept an offer → automatically creates a PurchaseTransaction
await api(`/offers/${offerId}/respond`, {
  method: "POST",
  body: JSON.stringify({ status: "accepted" }),
});
// The PurchaseTransaction is now in "offer_accepted" status
```

Escrow lifecycle (admin only)

Fund escrow:

```
await api(`/purchase-transactions/${id}/fund`, { method: "POST" });
```

- Pre-conditions: buyer and seller KYC verified, both wallets linked, listing verified
- Calls `SaleEscrow.openAndFund()` on-chain
- Status → `deposit_received`, escrow state → `funded`

Release escrow (complete sale):

```
await api(`/purchase-transactions/${id}/release`, { method: "POST" });
```

- Pre-conditions: escrow funded, listing has minted title token, buyer has wallet
- Two-step atomic operation:
 - i. Calls `SaleEscrow.release()` → funds to seller
 - ii. Calls `PropertyTitle.transferTitle()` → NFT to buyer
- Status → `completed`, listing → `sold`

Edge case: If escrow release succeeds but title transfer fails, the `ChainTransaction` for `title.transfer` is marked `failed`. The purchase is NOT marked `completed`. An admin must retry the title transfer manually.

Refund escrow (cancel sale):

```
await api(`/purchase-transactions/${id}/refund`, { method: "POST" });
```

- Calls `SaleEscrow.refund()` → funds back to buyer
- Status → `cancelled`

Purchase disputes

Same pattern as lease disputes:

```
// Open dispute (buyer, seller, or admin)
await api(`/purchase-transactions/${id}/dispute`, {
  method: "POST",
  body: JSON.stringify({ reason: "..." }),
});

// Resolve dispute (admin only)
await api(`/purchase-transactions/${id}/dispute/resolve`, {
  method: "POST",
  body: JSON.stringify({
    decision: "release", // "release" | "refund"
    note: "...",
  }),
});
```

- `release` → release escrow + transfer title (same as normal release)
- `refund` → refund escrow (same as normal refund)
- If no escrow was funded, dispute resolves off-chain only

Closing checklist

Admins can update a 5-item checklist during the `closing_review` phase:

```

await api(`/purchase-transactions/${id}/status`, {
  method: "PATCH",
  body: JSON.stringify({
    status: "closing_review",
    closingChecklist: {
      purchaseAgreement: true,
      inspection: true,
      financing: true,
      titleReview: true,
      settlementStatement: false,
    },
    note: "Pending final settlement statement",
  }),
});

```

Guard: Escrow-gated statuses (`deposit_received` , `closing_review` , `title_transfer_pending` , `completed`) **cannot** be set manually via PATCH — they must be reached through the dedicated escrow lifecycle endpoints (fund/release/refund). Returns 409 if attempted.

12. Rental Yield & Maintenance Records

Yield summary

```
const yield = await api(`/listings/${listingId}/rental-yield`);
```

Response shape:

```

{
  "listingId": "64abc...",
  "currency": "USD",
  "period": { "from": "2025-06-18", "to": "2026-06-18" },
  "grossRent": 18000,
  "maintenanceCost": 2400,
  "netIncome": 15600,
  "occupiedDays": 330,
  "occupancyRate": 0.904,
  "escrowHistory": [
    {
      "leaseId": "64def...",
      "status": "active",
      "escrowState": "active",
      "fundTxHash": "0x...",
      "settleTxHash": null
    }
  ],
  "annualizedYield": 5.2
}

```

- Rolling 12-month window

- `grossRent` : SUM Of $(\text{monthlyRent} / 30) \times \text{occupiedDays}$ across all qualifying leases
- `maintenanceCost` : sum of all maintenance records in the period
- `annualizedYield` : $(\text{netIncome} / \text{propertyValue}) \times 100$ (null if property has no price)
- Only listing owner or admin can access

Maintenance records

Create:

```
await api(`/listings/${listingId}/maintenance`, {
  method: "POST",
  body: JSON.stringify({
    leaseId: "64def...", // optional – link to specific lease
    type: "repair", // maintenance | repair | utility | tax | insurance | management | other
    amount: 500,
    currency: "USD",
    incurredAt: "2026-05-15",
    note: "Plumbing repair in unit 3B",
  }),
});
```

List:

```
const records = await api(
  `/listings/${listingId}/maintenance?type=repair&from=2026-01-01&to=2026-06-30&page=1&limit=20`
);
```

Response shape:

```
{
  "items": [
    {
      "id": "64abc...",
      "listing": "64def...",
      "lease": "64ghi...",
      "type": "repair",
      "amount": 500,
      "currency": "USD",
      "incurredAt": "2026-05-15T00:00:00Z",
      "note": "Plumbing repair in unit 3B",
      "createdAt": "2026-05-16T10:00:00Z"
    }
  ],
  "total": 1,
  "page": 1,
  "limit": 20
}
```

13. Chain Transactions (Audit Trail)

Every blockchain interaction is recorded in the `ChainTransaction` collection.

Operations

Category	Operations
Title	<code>title.mint</code> , <code>title.dispute</code> , <code>title.clear_dispute</code> , <code>title.revoke</code> , <code>title.transfer</code>
Lease Escrow	<code>lease_escrow.open_and_fund</code> , <code>lease_escrow.activate</code> , <code>lease_escrow.cancel</code> , <code>lease_escrow.release_deposit</code> , <code>lease_escrow.refund_deposit</code>
Sale Escrow	<code>sale_escrow.open_and_fund</code> , <code>sale_escrow.release</code> , <code>sale_escrow.refund</code>

Statuses

Status	Meaning
<code>submitted</code>	Transaction submitted to blockchain
<code>pending</code>	Awaiting mining
<code>mined</code>	Transaction mined (initial confirmation)
<code>confirmed</code>	Fully confirmed
<code>reverted</code>	Transaction reverted on-chain
<code>stale</code>	Transaction has not been mined within expected time
<code>reconciled</code>	Manually reconciled after failure
<code>failed</code>	Backend-side failure before or during chain call

Chain transaction record shape

```
{
  "id": "64abc...",
  "operation": "lease_escrow.open_and_fund",
  "status": "mined",
  "targetType": "lease",
  "targetId": "64def...",
  "contractAddress": "0x1234...",
  "txHash": "0xabcd...",
  "blockNumber": 12345678,
  "confirmedAt": "2026-06-04T10:01:30Z",
  "metadata": { "escrowId": "1", "leaseId": "64def..." },
  "createdBy": "64ghi...",
  "createdAt": "2026-06-04T10:00:00Z"
}
```

Frontend guidance

- Use `txHash` to link to block explorer

- Color-code statuses: green (mined/confirmed), yellow (pending/submitted), red (failed/reverted), gray (stale/reconciled)
- Poll for status changes on pending transactions

14. Notifications

Lease-related notification types

Type	Triggered when
lease.status_update	Any lease status change (propose, sign, fund, activate, complete, terminate, cancel, dispute, resolve)
purchase.status_update	Any purchase transaction status change (fund, release, refund, dispute, resolve)
rental_application.received	New rental application submitted
rental_application.status_update	Application status change (approve, reject, waitlist)

Notification object

```
{
  "id": "64abc...",
  "recipient": "64def...",
  "type": "lease.status_update",
  "title": "Escrow funded",
  "message": "The lease escrow has been funded on-chain.",
  "metadata": {
    "leaseId": "64ghi...",
    "status": "proposed",
    "escrowId": "1",
    "txHash": "0x..."
  },
  "readAt": null,
  "createdAt": "2026-06-04T10:00:00Z"
}
```

Reading & marking read

```
// List my notifications
const notes = await api("/notifications?page=1&limit=20");

// Mark as read
await api(`/notifications/${id}/read`, { method: "POST" });
```

15. Error Reference

Common error codes across lease/purchase endpoints

HTTP	Code	Meaning
400	Bad Request	Validation failed (e.g., missing wallet address)
401	Unauthorized	No or invalid auth token
403	Forbidden	Insufficient role or KYC not verified
404	Not Found	Lease/transaction not found or user is not a party
409	Conflict	Invalid state transition or duplicate action
502	Bad Gateway	On-chain call succeeded but event parsing failed
503	Service Unavailable	Blockchain integration not configured

Specific error scenarios

Scenario	Status	Message
Fund escrow without tenant signing	409	"Tenant has not signed the lease"
Fund escrow twice	409	"Escrow already funded"
Activate without funded escrow	409	"Escrow is not funded"
Complete a proposed lease	409	"Cannot complete a lease that is not active"
Landlord missing wallet	400	"The landlord must have a linked wallet address"
Tenant missing KYC	403	"The tenant must complete KYC verification..."
Date overlap	409	"Overlapping lease exists for this listing"
Dispute already open	409	"A lease in disputed status cannot be disputed again"
Opener responds to own dispute	403	"You cannot respond to a dispute you opened"
Manual set to escrow-gated status	409	"Cannot manually set status to ..."
Blockchain not configured	503	"Lease escrow integration is not configured"
Mainnet guard	403	"Lease escrow operations on Ethereum mainnet are disabled"

16. Recommended Frontend Patterns

Lease detail page structure

Lease #64abc...

Status: ACTIVE

Lease Details

Listing: ...

Landlord: ...

Tenant: ...

Rent: \$1,500

Deposit: \$3,000

Period: ...

Timeline (vertical stepper)

✓ Created

✓ Proposed

✓ Signed

✓ Escrow Funded [tx link]

✓ Activated [tx link]

⌚ Settled

Escrow Details

State: active | Contract: 0x... | Token: 0x...

Fund TX: [link] | Escrow ID: 1

Actions (role-dependent)

[Complete Lease] [Terminate Lease] [Open Dispute]

Role-based action visibility

Action	Tenant	Landlord	Admin
View lease	✓	✓	✓
Sign lease	✓	✗	✓
Propose lease	✗	✓	✓
Cancel lease	✓	✓	✓
Fund escrow	✗	✗	✓
Activate	✗	✗	✓
Complete	✗	✗	✓
Terminate	✗	✗	✓
Open dispute	✓	✓	✓
Respond to dispute	✓*	✓*	✓
Resolve dispute	✗	✗	✓

*Only the counterparty (not the opener) can respond.

Polling strategy

- **Active leases with pending escrow:** Poll `/leases/:id/escrow` every 30s until `onChain.state` matches expected state
- **Timeline:** Poll `/leases/:id/timeline` every 60s on the detail page
- **Notifications:** Poll `/notifications?page=1&limit=5` every 30s or use WebSocket if available

Token amount formatting

On-chain amounts are in base units (e.g., 18 decimals). To display:

```
// Convert base units to human-readable
function formatTokenAmount(baseUnits: string, decimals: number = 18): string {
  const value = BigInt(baseUnits);
  const divisor = BigInt(10 ** decimals);
  const whole = value / divisor;
  const fraction = value % divisor;
  return `${whole}.${fraction.toString().padStart(decimals, '0').replace(/0+$/, '') || '0'}`;
}

// Example: "15000000000000000000" → "1500.0" (with 18 decimals)
```

Block explorer links

```
const EXPLORER_BASE = "https://sepolia.etherscan.io"; // or mainnet

function txLink(txHash: string): string {
  return `${EXPLORER_BASE}/tx/${txHash}`;
}

function addressLink(address: string): string {
  return `${EXPLORER_BASE}/address/${address}`;
}
```